MOVE DIGITAL · INTERNAL PLANNING DOCUMENT

eKYC Platform: Production Review and Product Strategy

PREPARED FOR	Malcolm, Move Digital
DATE	2 September 2026
SCOPE	`MalcolmGov/ekyc` at commit `6595ce1`, reviewed for production readiness; `MalcolmGov/aria` (the Zara agent marketplace) reviewed as the platform the product will be built on
STATUS	Planning document, revision 2. Direction agreed: build on the Zara platform as a reusable, multi-tenant KYC/KYB product sold to business customers. No code has been changed in either repository.

CONTENTS

Cumulative: a sellable KYC agent after Phase 2 (six to seven weeks), KYC plus onboarding after Phase 3 (about eleven weeks), and the full KYC, KYB and onboarding product after Phase 4 (four to five months).

---

1. Verdict

> **The ekyc repository is not production grade and should not be offered to clients in its current state.** It is a well-presented prototype: the UI is broad and the stack is sensible, but the verification engine behind it is largely simulated, the API has almost no authentication, passwords are stored in plain text, and results can be forged by anyone who can reach the server.

The `PRODUCTION-READINESS.md` file in the repo says "PRODUCTION READY, zero additional development required". That document is not accurate. Of the twenty-one "production ready" claims it makes, the code supports four (multi-country list, Didit session creation, PWA shell, multi-language UI). The rest are either simulated, marketing copy, or contradicted by the code.

The good news is that the product idea is sound and the market position is real. The repo is best treated as a **functional specification and UI prototype** for the new build, not as a codebase to patch or port.

Agreed path: build the product on the Zara platform rather than as a standalone service. Zara already has tenancy, partner API keys, rentals, metering, an audit table, retention purge, WhatsApp and web channels, a partner console, guardrails, evals, CI and a deploy pipeline, and a small working Didit client. A platform-level verification service plus three rentable agents (KYC, KYB, Digital Onboarding) is the product; ekyc is archived as the prototype reference. A sellable KYC agent is realistic in six to seven weeks; KYC plus

onboarding in about eleven weeks; the full KYC, KYB and onboarding product in four to five months (Section 9).

What this product is. A reusable KYC/KYB capability that you on-sell to business customers. Each customer is a Zara tenant. They rent the agents, call the verification API with their partner key, or both. Zara Pay, when it launches, is one tenant of this product like any other, not its owner.

---

2. What was reviewed and how

- Full read of `server/` (5,300 lines), `shared/schema.ts`, client hooks, API client, routing, and all deployment config.
- Install, typecheck (`npm run check`) and dependency audit run against a fresh clone.
- Every HTTP route catalogued with its authentication state (Appendix A).
- `aria` reviewed for the agent package format, catalogue and suite model, tenancy and billing tables, the webhook and MCP integration rails, the runtime sidecar, and the existing Didit client in Zara Pay, so the platform plan is grounded in what actually exists there.

I could not test against live Didit or Twilio credentials, so vendor behaviour is assessed from the code and the vendors' published contracts, not from live calls.

---

3. What the repository actually is today

CAPABILITY THE REPO PRESENTS	WHAT THE CODE DOES	STATUS
Identity verification (document, liveness, face match) via Didit	Creates a Didit session when real credentials are present. On any failure it silently returns a fabricated "demo" session. Webhook status updates never persist because of a lookup bug.	`Partially real, unreliable`

CAPABILITY THE REPO PRESENTS	WHAT THE CODE DOES	STATUS
Business verification (KYB) with registry, AML, UBO and sanctions checks	No registry, AML or sanctions call is made anywhere. A regex checks the registration number format, then a function named `performIntelligentBusinessVerification` produces an "approved" status, a risk score and AML results. The Ballerine endpoint it tries first does not exist as coded.	`Simulated`
Uganda KYB onboarding with document upload	Applications and uploaded ID documents are held in a JavaScript `Map` in process memory. Everything is lost on restart. Approval endpoint is unauthenticated.	`Demo only`
WhatsApp verification links via Twilio	Works when Twilio is configured. The send endpoint is unauthenticated, so anyone can send messages from your Twilio account to any number.	`Real, exposed`
Compliance dashboard with PII masking and audit trail	Masking exists for individual sessions only. The "audit log" is `console.log` with a hardcoded user of `system_user`. Business sessions (directors, UBOs, AML results) are returned unmasked and unauthenticated.	`Cosmetic`
Role-based access control	One middleware (`requireAdmin`) protects one route. Every other write endpoint is open.	`Cosmetic`
Developer API, SDKs, API keys	Marketing pages describe `npm install @swifterid/sdk` , `sk_live_` keys and `dashboard.swifterid.com` . None of these exist. There is no API key model or tenant model in the schema.	`Marketing only`

CAPABILITY THE REPO PRESENTS	WHAT THE CODE DOES	STATUS
54 African countries supported	A seeded table of 54 country rows with the same two document types each. Country support in a KYC product means vendor coverage per document type, which is not modelled.	`List only`
Multi-language (15 languages), PWA, mobile-first UI	Present in the client.	`Real`
Analytics dashboard	Real queries over the sessions table. Currently populated with 50 randomly generated sample sessions in development.	`Real, thin`

4. Findings

Severity is judged by what happens if a paying client is put on this system today. File references point at the reviewed commit.

Critical: blocks any client use

`C1` **Passwords are stored and compared in plain text.** `server/storage.ts:76` compares `user.password === password`. Any database read exposes every credential. There is no registration route, so users are created by hand in the database.

`C2` **Verification results can be forged by anyone.** `POST /api/webhook/didit` (`server/routes.ts:1431`) accepts `{session_id, status: "completed"}` from any caller with no signature check and writes it to the session. The compliance dashboard will then show that person as verified. `PATCH /api/verification-sessions/:id` (`routes.ts:1391`), `PATCH /api/business-verification/sessions/:id` (`routes.ts:1827`) and `PATCH /api/kyb/uganda/applications/:id/status`

(`routes/ugandaKYB.ts:285`) are also unauthenticated and allow arbitrary status changes including approval.

C3 **The signed webhook can be bypassed with a header.** The "secure" webhook at `/api/webhooks/didit` skips signature verification when the request carries `x-demo-mode: true` (`routes.ts:549`). The webhook secret also has a hardcoded fallback value committed in source (`routes.ts:18`).

C4 **Personal data is readable without authentication.** Full business verification records including directors, shareholders, beneficial owners and AML results (`GET /api/business-verification/sessions` , `routes.ts:1852`); Uganda applicant names, phone numbers and districts (`routes/ugandaKYB.ts:250`); WhatsApp session records with phone number, IP address and user agent (`routes.ts:2422`); the full analytics export (`routes.ts:1310`); and the list of live test-access tokens (`server/index.ts:137`).

C5 **The system fabricates verification outcomes when vendors fail.** When Didit session creation fails, the API returns a demo session rather than an error (`routes.ts:190` to `routes.ts:520` , including leftover debug code `if (false && MOCK_MODE || ...)`). KYB "completion" (`routes.ts:2170`) catches any vendor error and falls through to the simulation, then records `status: completed` , a risk score and clean AML results. A client would have no way to distinguish a real approval from a simulated one.

C6 **Outbound messaging is open to abuse.** `POST /api/whatsapp/create-verification-link` (`routes.ts:2265`) is unauthenticated. Anyone can drive Twilio spend and send branded WhatsApp messages to arbitrary numbers.

C7 **Secrets and access artefacts are committed.** `.test-tokens.json` contains live test-access tokens with names and email addresses. `new_cookies.txt` contains a session cookie. Docs contain the Twilio Account SID, Meta Business Manager IDs and message SIDs. On production start the server creates two new test tokens automatically (`server/index.ts:85`).

High: must be fixed before a pilot

H1 **Didit webhook updates never persist.** The handler looks the session up correctly, then calls `updateVerificationSession(session.id.toString(), ...)` with the numeric id where the string session id is expected (`routes.ts:584`). The update matches nothing, and the code logs success anyway. Real verifications will sit at `pending` forever.

H2 **Session security.** `cookie.secure` is `false` , the session secret defaults to `dev-secret-change-in-production` , and the default in-memory session store is used despite `connect-pg-simple` being installed (`server/index.ts:20`).

H3 **Security middleware is installed but not wired.** `helmet` , `cors` , `express-rate-limit` and `compression` are dependencies and none is used. The global error handler responds and then rethrows (`server/index.ts:185`).

H4 **Uploaded identity documents are held in memory.** `routes/ugandaKYB.ts:76` . No object storage, no encryption, no retention, lost on restart.

H5 **No tenancy, API keys or customer model.** The schema has `users` , `verification_sessions` , `countries` , `business_verification_sessions` and `whatsapp_verification_sessions` . Nothing links a session to a paying customer. You cannot onboard a second client without them seeing the first client's data.

H6 **Engineering baseline is missing.** `npm run check` fails with 13 type errors. There are no tests, no CI, no linter and no migrations (schema is applied with `drizzle-kit push`). `npm audit` on production dependencies reports 36 vulnerabilities: 1 critical, 21 high, 12 moderate, 2 low.

H7 **Deployment is tied to Replit.** Callback URLs are built from `REPLIT_DOMAINS` , `ekyc-africa.com` is hardcoded in eleven places, `vercel.json` deploys the frontend only (the API is excluded), and there is no Dockerfile.

H8 **Compliance claims are not backed by the code.** No audit table, no consent capture, no retention or deletion, raw vendor payloads stored in `vendor_data` JSON, no per-tenant isolation, biometric processing and cross-border transfer not addressed (see Section 8).

Medium: should be fixed in the rebuild

- Two definitions of `GET /api/stats` (`routes.ts:1139` and `routes.ts:1736`); the second is unreachable.
- Session creation tries three different authentication schemes against Didit in sequence rather than using the documented one.
- Webhook handler loads every session into memory to find one.
- UBO threshold hardcoded at 25% (`routes.ts:1985`); South Africa's beneficial ownership register uses 5%.
- Brand inconsistency across pages: eKYC Africa, SwifterID and Move Digital.
- Repository hygiene: roughly 200 screenshots, nine pitch decks and a 700 KB app export in the tree; `attached_assets/` is served publicly by the API.

What is worth keeping

- The stack: TypeScript end to end, React, Express, Drizzle, PostgreSQL. Nothing exotic, easy to hire for.
- Didit as the primary document and biometric vendor. Hosted flow, webhook model and free tier suit an African SME product.
- The WhatsApp link pattern: a link sent to the applicant that opens a hosted verification flow. This is the right channel for the market and maps directly onto how Zara agents hand off.
- The KYB data model shape: business, directors, shareholders, UBOs, documents, risk score, compliance status. The fields are right even though nothing fills them honestly.
- The country registration-number patterns and the data masking utility are usable starting points.
- The client as a reference: 26 pages of flows, copy and i18n that show what the hosted pages and agent scripts need to cover. It is not ported; Didit hosts document capture and Zara hosts the rest.

---

5. What you are actually selling

Before designing the rebuild, the three product lines need precise definitions, because "agent" means something different in each.

PRODUCT	WHAT THE CLIENT GETS	WHAT THE "AGENT" DOES	WHAT IT MUST NEVER DO
`KYC Agent`	Individual identity verification for their customers: document, liveness, face match, optional government database check, sanctions and PEP screening, a decision and an evidence pack	Talks to the applicant on WhatsApp or web, explains what is needed, sends the secure verification link, follows up on drop-off, answers questions, and reports the outcome to the client's system	Collect ID numbers, selfies or documents in the chat itself. Verification always happens in the hosted, audited flow.
`KYB Agent`	Business verification: registry lookup, director and UBO identification, KYC of each UBO, sanctions and adverse media, a risk rating, and a case file a compliance officer can defend	Collects business details conversationally, runs the checks through the core, KYCs the directors via links, chases missing documents, and prepares the case for human decision	Approve on its own. KYB decisions carry human accountability and the agent's output is a recommendation with evidence.
`Digital Onboarding Flows`	A configurable, white-labelled onboarding journey for the client's customers:	Optional. A flow can run with or without an agent in front of it.	Run without a flow definition owned by the client.

PRODUCT	WHAT THE CLIENT GETS	WHAT THE "AGENT" DOES	WHAT IT MUST NEVER DO
	form steps, KYC, KYB, agreements, and a webhook back into their systems on completion		

Two design rules fall out of this:

1. **The verification engine and the agent are separate products.** The engine is an API with an audit trail. The agent is a Zara package that calls it. Clients who want no agent still buy the engine.
2. **Agents orchestrate; they do not verify.** This is also how the existing Zara packages are written: the onboarding-buddy workflow "never collects banking/card in chat", and the loan pre-qualifier returns indicative results, "never a credit decision". KYC and KYB agents must hold the same line or the compliance story collapses.

Pilot tenants are already in the group. Gaslite needs verified drivers. Zara needs verified retail partners before it hands them a partner dashboard. Both are honest pilots that exercise the KYC flow, the WhatsApp channel and the webhook-out integration before an external client depends on them. They are pilots of a product built for external customers, not the reason the product exists.

---

6. Target architecture on Zara

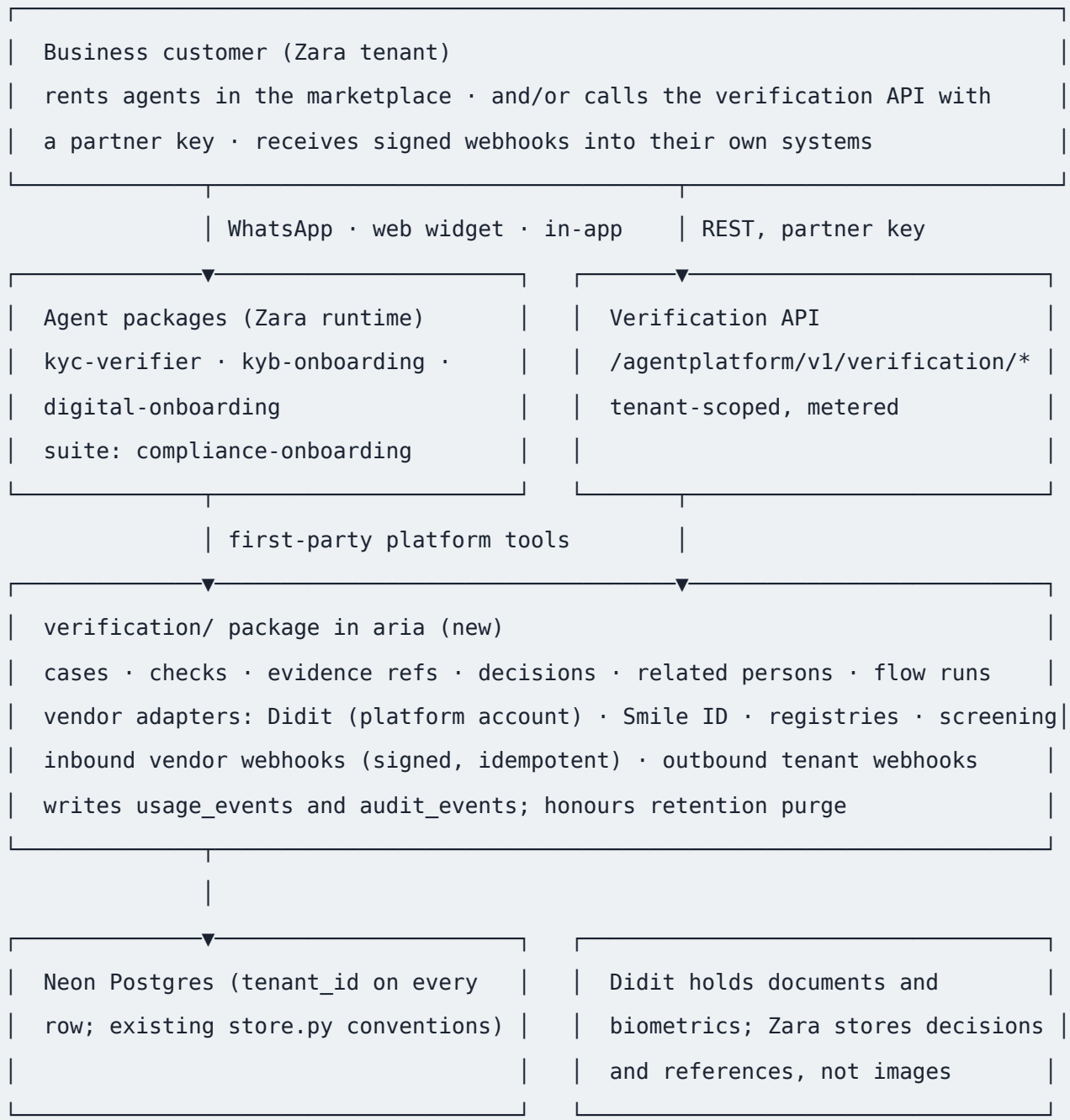
6.0 What Zara already provides

Every row below is shipped code in `aria`, not a plan. This is why building on Zara is faster and safer than a standalone service.

NEED	WHERE IT EXISTS IN ZARA	NOTES
Tenancy and customer identity	`partners`, `partner_keys`, `partner_members`, `reseller_subtenants` in `agent_platform/store.py`	Business customers are partners; API access is by hashed partner key
Commercial model	`rentals`, `deployments`, `usage_events`, `invoices`, `saas_subscriptions`, `partner_agent_prices`	Agent rental by tier plus metered usage already exists; per-check vendor cost is one more usage event type
Audit and retention	`audit_events` table; nightly `retention.py` purge job	Extend rather than invent
Channels	WhatsApp via Meta Cloud API, web embed widget, in-app	One deployment serves all three
Agent packaging	`zara.agent-package/v1` with typed tools, guardrails, evals; 500-SKU catalogue; suites	Three new families and one suite entry, no schema change
Per-tenant integrations	Actions of kind webhook and MCP with HMAC signing, SSRF guard	Outbound "verification complete" webhook to the customer's system uses the existing rail
White label	`white_label_configs`, brandless widget for white-labelled tenants	Hosted status pages and messages take the tenant's brand
Didit client	`vas-wallet/app/kyc/didit.py` and `webhook.py`	Correct signature check on raw bytes, correct status persistence; small, promoted to platform level in Phase 1
Engineering baseline	CI with 288 test files, deploy workflow, Neon Postgres, security pack, SA compliance calendar	ekyc has none of these

Two production-safety gaps in the current Zara Pay code must be closed when the client is promoted: the webhook skips signature verification when no secret is configured (`vas-wallet/app/kyc/didit.py:79`), and the signup flow activates a wallet without any KYC when Didit is not configured (`vas-wallet/app/flows/router.py:379`). Both are acceptable in development and must be impossible in production.

6.1 Shape



6.2 The verification package

A new `verification/` package in aria, mounted like the other routers in `server.py`, with its tables created through `store.py` conventions. It is platform code, not part of `vas-wallet`.

- **Tenant scope.** Every table carries `partner_id`. Cases are created by an agent acting for a tenant or by a partner-key API call. A tenant can only read its own cases; a test proves it.
- **Case model.** An `applicant` (person or business) has `cases`; a case has ordered `checks` (document, liveness, face match, government database, registry, sanctions, PEP, adverse media, UBO discovery); each check stores a typed `result` and an `evidence` reference (vendor session id, report reference, document pointer held by the vendor). A case ends in a `decision` with a named decider (system or a tenant user) and a reason. Related persons (directors, UBOs) link a business case to person cases.
- **Vendor adapters.** `IdentityVendor` (Didit first, Smile ID second), `RegistryVendor` (per country), `ScreeningVendor` (sanctions and PEP). Each returns a typed result or raises. No adapter may return a simulated success. Sandbox is a per-tenant flag visible on every record and every API response, and sandbox can never activate anything in a live tenant.
- **Didit account model.** One platform Didit account owned by Move Digital, with `vendor_data` carrying the tenant and case ids so webhooks route correctly, and per-check cost metered to the tenant as a `usage_events` row. Bring-your-own-account is a later enterprise option, not the default.
- **Webhooks.** Inbound vendor webhooks verified on raw bytes and idempotent on the vendor event id. Outbound "case decided" events to the tenant's system use the existing signed action rail, so the customer's integration guide is the one Zara already publishes.
- **Hosted pages.** A tenant-branded start page, status page and retry page. Document capture and liveness stay in Didit's hosted flow. The ekyc client is not ported.
- **Data minimisation.** Zara stores decisions, check results and references. Documents and selfies stay with the vendor. This keeps the POPIA surface small and the retention purge simple.

6.3 Vendor strategy

NEED	RECOMMENDED	WHY	NOTE
Document, liveness, face match	Didit, platform account	Hosted flow, webhooks, broad document coverage, free tier for early volume; client already exists in Zara	Move to the current session API and confirm the endpoint and header scheme against Didit's docs at build time
African government database checks (SA DHA, Nigeria NIN/BVN, Kenya IPRS, Ghana NIA)	Smile ID as second adapter	Direct government checks are where African KYC differentiates	Contractual and per-check cost; enabled per tenant
Sanctions, PEP, adverse media	Didit AML screening for the pilot; evaluate OpenSanctions (commercial licence) or ComplyAdvantage for KYB	One vendor for the pilot; KYB needs entity-level screening later	
Company registries	Adapter per country. South Africa first (CIPC data via an accredited data partner), then Kenya BRS, Nigeria CAC, Ghana ORC, Uganda URSB	API access differs by country and usually needs an agreement	Every registry adapter has a **manual review fallback**: an operator uploads the registry extract and attests. Honest assisted KYB beats fake automated KYB
Workflow engine (Ballerine)	Not used. The case state machine lives in the verification package	The ekyc integration called an endpoint that does not exist	Revisit only at scale

NEED	RECOMMENDED	WHY	NOTE
WhatsApp	Zara's Meta Cloud API number and templates	Already live with template approval and signature enforcement in CI	Twilio is not needed

6.4 Commercial model

- **Agent rental** at the existing tier prices in `agent_platform/config.py` (`kyc-verifier` pro, `kyb-onboarding` enterprise, `digital-onboarding` pro).
- **Per-check metering** as `usage_events`, invoiced through the existing invoice path, with a margin over vendor cost. Volume tiers are a pricing setting, not code.
- **API-only** customers use a partner key with no agent rental, billed on checks alone.
- **White label** and reseller sub-tenants use the existing `white_label_configs` and `reseller_subtenants` paths, so a bank or a fintech can resell onboarding under its own brand.

---

7. The three agents in the Zara marketplace

Zara already has the machinery this needs: a `zara.agent-package/v1` format with manifest, system prompt, knowledge, typed tools with `side_effects` and `auth_scope`, guardrails and evals; a catalogue of 500 SKUs across five markets; suites that bundle families; a runtime sidecar; and two integration rails (signed webhooks and MCP) documented at `docs/marketplace/CONNECT_YOUR_BACKEND.md`. There is no KYC or KYB family in the catalogue today, and the adjacent families (`onboarding-buddy`, `loan-prequalifier`, `sim-registration`, `policy-compliance`, `fraud-investigations`) all follow the "collect, explain, hand off, never decide" pattern.

7.1 New families

FAMILY ID	NAME	TIER	CHANNELS	WHAT IT DOES
`kyc-verifier`	Identity Verification Agent	pro	whatsapp, web, app	Explains what is needed, sends the secure verification link, tracks status, nudges on drop-off, answers document questions, reports outcome
`kyb-onboarding`	Business Verification Agent	enterprise	whatsapp, web	Collects business identity conversationally, opens a business case, triggers registry and screening checks, sends KYC links to each director and UBO, chases documents, hands a prepared case to a human
`digital-onboarding`	Digital Onboarding Agent	pro	whatsapp, web, app	Runs the tenant's configured onboarding flow end to end: form steps, KYC or KYB as steps, agreements, and a signed webhook into the tenant's systems on completion

All three are built for the `africa` market first with `en`, `zu`, `af`, `fr`, `sw` in the manifest, and `compliance: ["popia", "fica"]`. Market variants follow the existing `{market}-{family}` convention.

7.2 A `compliance-onboarding` suite

One entry in `agent_platform/suites.py`, no schema change: `kyc-verifier`, `kyb-onboarding`, `digital-onboarding`, `policy-compliance`, `fraud-investigations`. Sector: "Banks, lenders, insurers, fintech and any regulated onboarding". This gives the marketplace a compliance vertical alongside the insurance-ops suite shipped in August.

7.3 Tools

The verification package exposes first-party platform tools to the runtime (the same mechanism the existing families use), scoped to the renting tenant. They follow the loan pre-qualifier package's discipline:

TOOL	SIDE_EFFECTS	PURPOSE
`explain_requirements(country, document_type?)`	read-only	Authoritative checklist for th applicant's cou
`start_kyc(applicant_ref, channel, language)`	write	Creates a case returns the hos link. Never acc identity data a arguments.
`get_case_status(case_ref)`	read-only	Status and nex action, no PII ir response
`start_kyb(business_name, registration_number, country)`	write	Opens a busine case and kicks registry lookup
`add_related_person(case_ref, role, contact)`	write	Registers a dire or UBO and trig their KYC link
`request_document(case_ref, document_type)`	write	Adds an outstanding document requ and returns an upload link

TOOL	SIDE_EFFECTS	PURPOSE
`run_flow_step(flow_run_ref, step_id, answers)`	write	Advances a tenant's onboarding flo used by `digital-onboar` only
`escalate(case_ref, reason)`	write	Hands off to th tenant's compliance de

7.4 Guardrails and evals

- The agent never asks for, acknowledges or stores an ID number, date of birth, selfie or document image in chat. If a user sends one, the agent does not repeat it back and redirects to the secure link. This must be a post-reply check in the guardrail layer, not only a prompt instruction.
- The agent never states that someone "is verified" or "is approved" unless `get_case_status` returned that state, and never for KYB.
- Evals: at least one case per rule above, in `whatsapp` and `web`, in English and one other market language, using the existing `says_any` / `says_none` eval format.

7.5 Onboarding flows

A `flow_definitions` table per tenant: an ordered list of steps with types `form`, `kyc`, `kyb`, `document`, `agreement`, `review`, `webhook`. A `flow_runs` table tracks an applicant through a definition and is resumable from a link. The `digital-onboarding` agent drives a run conversationally; the same run can also be completed on the hosted pages without an agent. A tenant configures a definition in the partner console, gets a link or the widget, and receives a signed webhook when a run completes. This is what "Digital Onboarding flows for business customers" means concretely.

7.6 Delivery

Packages live in `data/agents/` as `africa-kyc-verifier.agent.json`, `africa-kyb-onboarding.agent.json` and `africa-digital-onboarding.agent.json`, validated by `scripts/marketplace/validate_package.py`, with a build note in `docs/marketplace/` matching `INSURANCE_OPS_SUITE.md`. The partner console gets a Verification tab: cases, review queue, flow builder, and per-check usage.

---

8. Compliance and regulatory requirements

This is not legal advice. It is the list of obligations the product must be built to satisfy, to be confirmed with counsel before the first regulated client.

South Africa (first market)

- **FICA.** Accountable institutions must perform customer due diligence, identify beneficial owners, screen against sanctions lists and keep records for five years. Your clients are the accountable institutions; your product must give them an evidence pack that survives an FIC inspection.
- **Beneficial ownership.** The CIPC register uses a 5% threshold. The threshold must be a per-jurisdiction setting.
- **POPIA.** Biometric data is special personal information (section 26) and requires consent or another authorisation; you need an Information Officer, a lawful-basis record per processing purpose, and a basis for cross-border transfer (section 72) because Didit and most screening vendors process outside South Africa, and Zara's own database is Neon in the EU. Data subjects can request access and deletion.
- **Retention and deletion.** Per-tenant policy, enforced by a job, with a deletion record.

Other markets in the current country list

- Nigeria: Nigeria Data Protection Act 2023 and CBN KYC tiers; BVN/NIN access is licensed.
- Kenya: Data Protection Act 2019, registration as a data processor, IPRS access via licensed partners.
- Ghana: Data Protection Act 2012, NIA Ghana Card verification via partners.
- Uganda: Data Protection and Privacy Act 2019; NIRA and URA access via agreements.

Product controls that follow

- Consent capture at the start of every hosted flow, stored as evidence with timestamp, IP and text version.
- Data residency setting per tenant, honoured by object storage region.
- Human decision on every KYB case; system decisions on KYC only where the tenant has opted in and the vendor confidence is above a tenant threshold.
- A DPA (data processing agreement) template and a security overview for client procurement. Zara already has `agent_platform/security_pack.py`; extend it.
- Independent penetration test before the first external client.

---

9. Roadmap

Effort is indicative, for one to two engineers working in `aria`. Each phase has exit criteria; a phase is not done until they pass. Phases 2 and 3 can overlap once the Phase 1 API is stable.

PHASE	GOAL	WORK	EXIT CRITERIA
`0. Close ekyc`	Stop the exposed prototype being a liability	Rotate the Didit, Twilio and session secrets it holds. Take the deployment offline or put every write and PII endpoint behind authentication.	Nothing reachable on the internet serves ekyc's open endpoints. No live secrets in the tree.

PHASE	GOAL	WORK	EXIT CRITERIA
		Remove committed tokens and cookies from the repo and its history. Archive the repo as the prototype reference.	
`1. Verification service`	A tenant-scoped, metered verification API on Zara with real Didit results	`verification/` package. Case model with tenant scope. Didit adapter promoted from Zara Pay to platform level, production-safe config (no signature skip, no sandbox activation). Inbound webhook, idempotent. Partner-key API. Usage and audit events. Hosted start, status and retry pages. Outbound "case decided" event on the existing action rail. Tests for tenancy isolation, signature verification and idempotency.	Gaslite and Zara partner onboarding tenants complete real verifications end to end from a partner-key call. Usage rows and audit rows written. CI green.
`2. KYC agent`	`kyc-verifier` rentable in the marketplace	Package, tools, guardrails, evals. Suite entry. Console Verification tab	Package passes `validate_package.p` and evals. A rented agent completes a

PHASE	GOAL	WORK	EXIT CRITERIA
		with cases list. WhatsApp and widget flows.	real verification fro WhatsApp and fron the widget for a pil tenant. Post-reply guardrail blocks ID data in chat.
`3. Digital Onboarding agent`	Configurable, white-labelled onboarding journeys	`flow_definitions` and `flow_runs`. Flow builder in the console. Step renderers on hosted pages. Agreements step with an e-signature record. Consent capture as evidence. `digital-onboarding` package. Retention honoured for flow data.	A tenant configure a flow without engineering help, a applicant complete it on WhatsApp wit KYC as a step, and the tenant receives the signed webhoo
`4. KYB agent`	Honest business verification with human decision	Registry adapter interface, South Africa first with manual-review fallback. Screening adapter. UBO discovery with per-jurisdiction threshold. Related-person KYC links. Review queue in the console with decision and reason. Evidence pack export. `kyb-onboarding` package.	A South African company is onboarded with a registry extract, screened directors and UBOs, and a signed decision. No code path produce a result without a vendor response o a human attestation.

PHASE	GOAL	WORK	EXIT CRITERIA
`5. Trust and scale`	Sell to regulated clients	Penetration test and remediation of the verification surface. DPA and security pack additions. Smile ID adapter. Second registry country. Load test.	Pen test with no open high findings First external regulated client signed.

Cumulative: a sellable KYC agent after Phase 2 (six to seven weeks), KYC plus onboarding after Phase 3 (about eleven weeks), and the full KYC, KYB and onboarding product after Phase 4 (four to five months).

10. Definition of production ready

The product is ready to offer to a client when all of the following hold. Use this as the release gate for the Zara verification service and the three agents; it replaces ekyc's `PRODUCTION-READINESS.md`.

- [ ] Every route is authenticated and scoped to a tenant; an automated test proves tenant A cannot read tenant B.
- [ ] Passwords hashed with argon2; API keys hashed; secrets only from environment; no secrets in git history.
- [ ] Every inbound webhook verified on raw bytes and idempotent; no bypass paths.
- [ ] No code path returns a verification, screening or registry result that did not come from a vendor response or a recorded human attestation. Sandbox results are labelled on every record and every API response, and sandbox can never activate anything in a live tenant.

- [ ] Append-only audit events for every read of PII and every state change, with actor, time and origin.
- [ ] Consent captured and stored per applicant; retention policy enforced by a scheduled job; deletion produces a record.
- [ ] Documents and vendor payloads stored encrypted in region-pinned object storage, never in the database or process memory.
- [ ] CI blocks merge on typecheck, lint, tests and dependency audit (no critical or high).
- [ ] Containerised deployment with migrations, health checks, structured logs, rate limiting and security headers.
- [ ] Independent penetration test with no open high findings.
- [ ] DPA, privacy notice and security overview available for client procurement.
- [ ] Marketing pages describe only what exists.

---

11. Decisions

Taken

1. **Build on Zara, not standalone.** The verification service is a platform package in `aria`; the product is sold as rentable agents plus a partner-key API. ekyc is archived as the prototype reference.
2. **Reusable, multi-tenant, on-sold.** Every business customer is a Zara tenant. Zara Pay, when it launches, is one tenant.

Still needed from you

1. **Brand.** The product name customers see in the marketplace and on hosted pages. eKYC Africa, SwifterID, or a Zara-branded name such as "Zara Verify".
2. **Didit account model.** Recommended: one platform account owned by Move Digital with per-tenant metering. Confirm, or specify bring-your-own-account for enterprise tenants from day one.
3. **Pricing.** Confirm agent tiers (`kyc-verifier` pro, `kyb-onboarding` enterprise, `digital-onboarding` pro) and the per-check margin over vendor cost.
4. **First external market and vertical.** South Africa lenders and fintech is the natural fit with FICA and the banking suite. Confirm.

5. **KYB honesty line.** Approve "assisted KYB with human decision" for Phase 4 rather than promising automated registry checks where API access is not yet contracted.
6. **Vendor conversations.** Approve opening commercial discussions with Didit (production tier), Smile ID and a South African company data partner.
7. **Pilots.** Approve Gaslite driver onboarding and Zara partner onboarding as the two pilot tenants.

Appendix A: route inventory and authentication state

ROUTE	METHOD	AUTH TODAY	DATA EXP OR CHANG
`/api/auth/login` `/logout` `/me`	POST/GET	session	credent (plain te compar
`/api/verification-sessions`	POST	admin	creates session
`/api/verification-sessions`	GET	none	all sessi (maske
`/api/verification-sessions/:id`	GET	none	session detail (maske
`/api/verification-sessions/:id`	PATCH	none	status, confide
`/api/webhooks/didit`	POST	HMAC, bypassable	session status (update no-op)
`/api/webhook/didit`	POST	none	session status

ROUTE	METHOD	AUTH TODAY	DATA EXP OR CHANG
`/api/didit/create-session`	POST	none	creates session
`/api/didit/console-data/:id` `/console-sessions` `/session-status/:id` `/discover` `/callback`	GET/POST	none	vendor
`/api/stats` `/api/analytics/comprehensive` `/api/analytics/export`	GET	none	aggrega and full export
`/api/business-verification/sessions`	POST/GET	none	full KYB records UBOs, A
`/api/business-verification/sessions/:id`	GET/PATCH	none	full reco arbitrar update
`/api/business-verification/sessions/:id/complete`	POST	none	simulate approva
`/api/kyb/uganda/onboard`	POST	none	in-mem applicat with ID
`/api/kyb/uganda/status/:ref`	GET	none	status
`/api/kyb/uganda/applications`	GET	none	all applicar
`/api/kyb/uganda/applications/:id/status`	PATCH	none	approve reject
`/api/kyb/uganda/validate-tin` `/validate-nid`	POST	none	regex o
`/api/whatsapp/create-verification-link`	POST	none	sends WhatsA via Twili

ROUTE	METHOD	AUTH TODAY	DATA EXP OR CHANG
`/whatsapp-verify/:token` `/api/whatsapp/verify/:token`	GET	token	applicar session
`/api/whatsapp/session/:id`	GET	none	phone, user ag
`/api/maintenance/enable` `/disable` `/api/emergency-disable-maintenance`	POST/GET	none	mainter flag
`/api/test-tokens` `/create` `/deactivate` `/cleanup`	GET/POST	none	all test tokens
`/attached_assets/*` `/test-manager.html` `/admin-maintenance.html` `/emergency-access.html`	GET	none	screens admin p

Appendix B: build and audit output

```
npm run check      13 type errors (client 7, server 6); build script bypasses tsc so t
npm audit --omit=dev
                   36 vulnerabilities: 1 critical (fast-xml-parser), 21 high (incl. ws
tests              none
CI                 none
lint               none
migrations         none (drizzle-kit push)
Dockerfile         none
```

Unused production dependencies observed: `helmet`, `cors`, `express-rate-limit`, `compression`, `connect-pg-simple`, `passport`, `passport-local`, `openai`, `@google-`

`cloud/storage`, `@uppy/*`, `@ballerine/web-ui-sdk`, `twilio` (raw fetch is used instead).